FOREX TRADING SIGNAL AI AGENT

Professional Project Report

EUR/USD • Technical Indicators • Machine Learning • Model Validation • Power BI

Prepared by

Ramu Suda

An exploratory analytics prototype for studying currency-price data, generating directional signals, comparing machine-learning models, and communicating results through dashboards.

IMPORTANT: This project is for research and educational purposes. The reported results do not establish a profitable trading strategy and should not be interpreted as financial advice.

Report scope: project structure, source data, feature engineering, signal generation, model comparison, walk-forward validation, backtesting, reporting deliverables, limitations, and recommended next steps.

Executive Summary

The Forex Trading Signal AI Agent is an end-to-end exploratory analytics project focused on EUR/USD price data. It brings together data inspection, technical-indicator engineering, rule-based signal generation, machine-learning classification, model evaluation, backtesting, and reporting. Python is used for data preparation and modelling; CSV files provide transparent intermediate outputs; an Excel workbook consolidates the analysis; and a Power BI dashboard communicates price behaviour, signals, model accuracy, and validation results.

The modelling experiments compared Random Forest, a reduced-feature Random Forest, XGBoost, an LSTM model, and a majority-class baseline. On a chronological holdout set of 90 observations, XGBoost achieved 53.33% accuracy, followed by the full-feature Random Forest at 52.22%. The XGBoost walk-forward experiment averaged 55.56% accuracy across four folds, compared with 50.56% for the baseline. These results are modest and are based on a small dataset, so they should be treated as exploratory evidence rather than proof of reliable predictive power.

The simple backtest was not profitable under the recorded assumptions: 277 periods were active, with 126 winning and 150 losing outcomes. The reported gross cumulative return was -0.4414%, and net cumulative return was -2.9471% after assuming a one-pip cost per active period. This distinction matters: a model can achieve accuracy slightly above a baseline and still fail to produce a profitable strategy after costs, sizing, and execution constraints.

Headline conclusion: XGBoost was the strongest tested classifier in the recorded experiments, but performance is limited and the current backtest is negative. Further data, realistic costs, stronger validation, and risk controls are required before any real-world use.

Key project facts

Area	Recorded project detail
Market	EUR/USD hourly price data
Source data	500 price rows; 1,000 Monte Carlo scenarios; 150 trade-log records
Feature engineering	Technical-indicator output contains 500 rows and 24 columns
Holdout evaluation	450 usable rows; chronological split of 360 train / 90 test
Best holdout result	XGBoost, 20 features: 53.33% accuracy
Walk-forward result	Mean XGBoost accuracy 55.56%; baseline 50.56%
Backtest result	Gross -0.4414%; net -2.9471% under the recorded cost assumption
Reporting	Excel workbook with 12 sheets; Power BI dashboard and supporting CSV files

Report Guide

This report is organized to explain both what was built and how its results should be interpreted. The sections below cover the analytical workflow, model experiments, validation, dashboard, and future work.

Section	Contents
1. Project overview	Purpose, objectives, scope, and intended users
2. Architecture and workflow	How data moves through preparation, modelling, evaluation, and reporting
3. Data inventory	Source datasets, size, time range, and supporting outputs
4. Feature engineering and signals	Technical indicators and directional signal generation
5. Modelling approach	Random Forest, XGBoost, LSTM, baseline, and chronological split
6. Results and comparison	Holdout accuracy and interpretation
7. Walk-forward validation	Fold-by-fold results and average performance
8. Backtesting	Trade outcomes, gross/net returns, and cost sensitivity
9. Reporting and dashboard	Excel workbook, Power BI visuals, and repository
10. Limitations and risks	Small sample, model uncertainty, execution and financial risk
11. Recommendations	Specific next steps for improving evidence and implementation
Appendix	File inventory and glossary

1. Project Overview

1.1 Background

Foreign-exchange prices respond to changing macroeconomic expectations, liquidity, market sentiment, and short-term order flow. Technical indicators can summarize historical price and momentum behaviour, while machine-learning classifiers can be tested for their ability to distinguish future directional outcomes. Neither indicators nor models guarantee that future movements can be predicted accurately.

1.2 Problem statement

The project investigates whether a structured pipeline can transform historical EUR/USD prices into technical features, produce directional trading signals, compare candidate classification models, and present the resulting evidence in a form that is understandable to a data analyst or reviewer.

1.3 Objectives

- Inspect and validate the source CSV datasets and document their basic structure.
- Calculate technical indicators and create a reusable feature dataset.
- Generate and save trading-signal and backtest outputs.
- Train and compare Random Forest, XGBoost, and LSTM experiments against a simple majority-class baseline.
- Use a chronological holdout and a four-fold walk-forward experiment to examine performance over time.
- Record feature importance and SHAP-based explanations for the XGBoost model.
- Publish results in an Excel workbook and a Power BI dashboard, with a clear disclaimer about limitations.

1.4 Scope

The scope is an educational prototype and retrospective analysis of a short EUR/USD hourly dataset. It is not a broker-connected trading bot, not a live execution system, and not evidence that a strategy will remain effective in future market conditions. The Monte Carlo scenarios and trade log are included as project data assets; the performance figures in this report are the recorded results from the model and backtest experiments.

1.5 Intended outcome

The intended outcome is a reproducible project portfolio artifact demonstrating practical skills in Python data analysis, feature engineering, supervised learning, time-aware validation, basic backtesting, explainability, Excel reporting, Power BI dashboard design, and Git/GitHub project organization.

2. System Architecture and Workflow

The pipeline is organized as a sequence of analytical stages. Intermediate CSVs and saved model artifacts make the work easier to inspect and reproduce.

Stage	Input	Action	Output / evidence
1. Data inspection	Three source CSVs	Check dimensions, fields, time coverage, and data usability	Data checks and source inventory
2. Feature engineering	Hourly price data	Calculate technical indicators and derived columns	forex_technical_indicators.csv
3. Signal generation	Price/indicator features	Generate directional signal labels according to project logic	forex_trading_signals.csv
4. Backtest	Signals and price movements	Evaluate recorded outcomes and assumed transaction costs	forex_backtest_results.csv
5. Model training	Usable labelled features	Train Random Forest, XGBoost, and LSTM experiments	Saved models and result CSVs
6. Evaluation	Chronological test data	Compare accuracy with baseline and run walk-forward folds	Comparison and validation reports
7. Explainability	Trained XGBoost model	Export feature importance and SHAP summaries	Importance CSVs and PNG charts
8. Communication	Analysis outputs	Consolidate results in Excel and Power BI	Workbook, dashboard, README

Workflow interpretation

The separation between raw data, engineered features, model artifacts, and reports helps reduce confusion about where results originate. A result should always be traceable to the data version, feature set, split method, model configuration, and assumptions used to produce it. The current pipeline demonstrates these stages, but reproducibility should be strengthened by recording package versions, random seeds, and a formal configuration file.

3. Data Inventory and Preparation

3.1 Source datasets

File	Recorded dimensions / role	Notes
forex_price_data.csv	500 rows × 9 columns	EUR/USD hourly prices from 2023-01-01 00:00 through 2023-01-21 19:00
mc_scenarios.csv	1,000 rows × 8 columns	Monte Carlo scenario data supplied with the project
trade_log.csv	150 rows × 15 columns	Trade-log records supplied with the project

3.2 Coverage and sample size

The hourly price file spans approximately three weeks. This is a very small sample for a market that changes across economic regimes, volatility environments, sessions, and liquidity conditions. The short period may not represent the range of conditions the model would face after deployment. A high number of engineered columns does not compensate for a small number of independent time observations.

3.3 Derived datasets and artifacts

Artifact	Purpose
forex_technical_indicators.csv	Engineered feature table; 500 rows and 24 columns
forex_trading_signals.csv	Saved directional signal output
forex_backtest_results.csv	Recorded backtest series; 499 rows
random_forest_model.joblib	Saved Random Forest model artifact
xgboost_model.joblib	Saved XGBoost model artifact
lstm_model.keras	Saved LSTM model artifact
lstm_scaler.joblib	Saved scaling object used in the LSTM workflow
Model comparison / tuning / walk-forward CSVs	Evaluation evidence for comparing configurations and folds
SHAP outputs and feature importance CSVs	Model explanation artifacts

3.4 Data quality considerations

- Ensure timestamps are parsed consistently and sorted in ascending order before any time-based split.
- Check missing values created by rolling indicators at the beginning of the series; do not silently fill future-derived information backward.
- Confirm labels represent the intended future horizon and are aligned so features at time t never include information from t+1 or later.
- Keep all preprocessing steps, including scaling and feature selection, fit only on the training portion of each split.
- Record data provenance and licensing before sharing the source files publicly.

4. Technical Indicators and Signal Generation

4.1 Feature engineering

Technical indicators convert raw price history into summaries of trend, momentum, and price movement. The project's engineered table contains 24 columns across 500 observations. The Power BI report uses the RSI_14 and MACD/MACD_Signal fields for visual inspection, alongside price and signal history.

4.2 Indicators used in reporting

Indicator / field	Typical interpretation	Caution
RSI_14	A 14-period momentum oscillator commonly used to inspect relative strength	Thresholds are not universal; extreme readings can persist in trends
MACD	Difference between moving-average-based trend measures	Can lag fast market changes and may whipsaw in sideways conditions
MACD_Signal	Smoothed signal line used alongside MACD	Crossovers are descriptive, not guarantees
Close	Closing price used to show the observed price path	A price chart alone does not establish tradability
Trading_Signal	Recorded signal category used for counts and filtering	Interpretation depends on the project's signal-generation rules

4.3 Signal generation

The pipeline saves its signal output to forex_trading_signals.csv. In the Power BI dashboard, a column chart displays the count of Trading_Signal categories, and a slicer allows the viewer to filter the report by signal. The exact trading rules should be documented alongside the code before interpreting any category as a buy, sell, or hold recommendation.

4.4 Avoiding leakage

Indicators must be calculated using only observations available at the signal timestamp. If a label is based on a future return, the target must be shifted appropriately and the final row without a future outcome handled explicitly. Any parameter tuning, feature selection, or scaling must be repeated inside each training fold rather than performed once on the full dataset.

5. Machine-Learning Methodology

5.1 Prediction setup

The project compares models on a directional classification task. The reported holdout experiment used 450 usable observations, split chronologically into 360 training rows and 90 test rows. A time-ordered split is more appropriate than random shuffling for sequential financial data because it tests on observations later than those used for training.

5.2 Candidate models

Model	Role in the experiment	Interpretation
Random Forest (20 features)	Tree-ensemble reference model using the full feature set	Provides a non-linear ensemble benchmark
Random Forest (8 features)	Reduced-feature experiment	Tests whether a smaller feature subset retains performance
XGBoost (20 features)	Gradient-boosted tree model	Best recorded holdout accuracy among the listed candidates
LSTM (20 features)	Sequence-oriented neural network experiment	Tests a recurrent model, but results are weak on this dataset
Majority-class baseline	Predicts the most frequent class	Simple reference to check whether model accuracy exceeds a naive strategy

5.3 Hyperparameter tuning and explanation

The project contains XGBoost tuning results and walk-forward outputs. The recorded tuning experiment did not outperform the original XGBoost configuration, so the original remains the strongest of the tested configurations in the current report. Feature importance CSVs were produced for Random Forest and XGBoost. SHAP values and summary/bar plots were also generated for XGBoost to help explain which features influenced model predictions.

5.4 Model interpretation

Feature importance and SHAP summaries explain model behaviour within the fitted dataset; they do not establish causation or guarantee that the most important feature will remain important in future periods. Correlated indicators can divide or distort importance scores, and explanation quality depends on the model, background data, and feature distribution.

6. Holdout Results and Model Comparison

The table below reproduces the recorded holdout accuracy figures. Accuracy is the percentage of test observations classified correctly; it does not measure trade profitability, expected return, drawdown, or risk-adjusted performance.

Model	Feature set	Holdout accuracy
XGBoost	20 features	53.33%
Random Forest	20 features	52.22%
Random Forest (reduced)	8 features	51.11%
Majority-class baseline	Baseline	47.78%
LSTM	20 features	47.78%

6.1 What the results suggest

- XGBoost achieved the highest recorded holdout accuracy, at 53.33%.
- The full-feature Random Forest was 1.11 percentage points below XGBoost.
- The reduced-feature Random Forest was slightly weaker than the full-feature version, suggesting that the selected smaller feature set did not improve this split.
- LSTM matched the majority-class baseline at 47.78% and showed a tendency to predict the up class in 84 of 90 test cases, indicating weak class balance in its predictions.
- The differences are small and may not persist on a larger or different time period.

6.2 Why accuracy is insufficient

A directional classifier can be correct often but still lose money if losing trades are larger than winning trades, costs are high, signals overlap, or the model enters and exits at unrealistic prices. Evaluation should include class-specific precision and recall, balanced accuracy, confusion matrices, and—only after a fully specified trading rule—net returns, drawdown, turnover, and risk-adjusted metrics.

Do not describe the 53.33% holdout accuracy as a proven trading edge. It is a small-sample classification result and needs independent validation.

7. Walk-Forward Validation

Walk-forward validation evaluates the model across multiple time-ordered folds. Each fold tests a later segment than the corresponding training data, providing a more informative view of temporal variability than a single split. The recorded project contains four XGBoost validation folds.

Fold	XGBoost accuracy	Baseline accuracy	XGBoost balanced accuracy
1	48.89%	51.11%	49.31%
2	60.00%	55.56%	60.50%
3	57.78%	46.67%	58.04%
4	55.56%	48.89%	54.64%
Mean	55.56%	50.56%	55.62%

7.1 Interpretation

- Mean XGBoost accuracy was 55.56%, compared with 50.56% for the baseline, a difference of 5.00 percentage points.
- Performance varied materially across folds: 48.89% in Fold 1 and 60.00% in Fold 2.
- Fold 1 was below the baseline, while Folds 2–4 exceeded their corresponding baseline accuracy.
- Mean balanced accuracy was 55.62%, which is useful because it accounts for performance across both classes more evenly than raw accuracy.
- Four folds over a very short historical period are not enough to establish stability across market regimes.

7.2 Recommended validation improvements

Use a larger historical sample, preserve strictly chronological boundaries, and reserve a final untouched test period that is not used for model selection. If the model or trading rules are repeatedly adjusted after inspecting results, the test period ceases to be independent. Where possible, repeat the experiment across different market regimes and report fold-level metrics rather than only an average.

8. Backtest Results and Economic Interpretation

The recorded backtest evaluates the signal series over 499 rows. The summary below is the project's recorded outcome, including the assumed transaction-cost treatment. It should be read as a simple prototype backtest rather than a complete execution simulation.

Metric	Recorded result	Interpretation
Active periods	277	Periods in which the backtest had an active signal/position
Winning outcomes	126	Recorded wins
Losing outcomes	150	Recorded losses
Gross cumulative return	-0.4414%	Return before the recorded transaction-cost deduction
Net cumulative return	-2.9471%	Return after assuming 1 pip cost per active period

8.1 Main conclusion

The recorded net cumulative return is negative. Therefore, the current backtest does not support a claim that the strategy is profitable. The difference between gross and net returns also illustrates how frequent trading and transaction costs can materially affect results.

8.2 Important implementation caveats

- The cost assumption is simplified; actual spread, commission, slippage, and financing/swap costs vary by broker, pair, time, and trade size.
- A realistic backtest should define signal timing, entry price, exit rule, stop-loss/take-profit logic, position sizing, and treatment of simultaneous or consecutive signals.
- Returns should be checked for compounding consistency, exposure, leverage, drawdown, and the treatment of periods without a position.
- Avoid look-ahead bias: a signal derived from a candle close should not assume execution at that same close unless the execution assumption is explicitly justified.
- The recorded win/loss counts do not by themselves explain average win size, average loss size, expectancy, or tail risk.

The backtest is negative after the recorded cost assumption. No live trading should be based on these results without substantial additional testing and risk controls.

9. Reporting, Visualization, and Project Deliverables

9.1 Excel workbook

The workbook `Forex_Trading_Signal_AI_Report.xlsx` consolidates the project into 12 sheets: Price Data, Technical Indicators, Trading Signals, Backtest Results, Model Comparison, Walk Forward Folds, Walk Forward Summary, XGBoost Tuning, RF Fair Comparison, XGBoost Importance, RF Importance, and LSTM Results. This provides a convenient review surface for the underlying data and evaluation outputs.

9.2 Power BI dashboard

The Power BI report contains a set of visuals for monitoring the price series and summarizing the analysis. The current dashboard includes the following components:

- Price-count and maximum-closing-price KPI cards.
- EUR/USD closing-price line chart and a DateTime range slicer.
- Trading-signal count chart and Trading_Signal slicer.
- RSI_14 line chart and MACD/MACD_Signal line chart.
- Model accuracy comparison chart for XGBoost, Random Forest variants, majority baseline, and LSTM.
- Four-fold XGBoost-versus-baseline walk-forward accuracy chart.
- KPI cards for average XGBoost walk-forward accuracy (55.56%), baseline accuracy (50.56%), and improvement (5.00 percentage points).
- Walk-forward fold table and a validation summary/disclaimer text box.

9.3 GitHub portfolio

The project is organized in a Git repository with folders for Data, Excel, PowerBI, Presentation, Python, and Reports, plus a README. The README includes a project overview, technology badges, project structure, results, a dashboard screenshot, and a disclaimer. The repository is available at: https://github.com/ram-analyst/Forex_Trading_Signal_AI_Agent

9.4 Why the reporting layer matters

The Excel workbook supports detailed inspection, while Power BI provides a quick visual summary for a reviewer. GitHub provides a shareable project structure and code history. Together, these deliverables demonstrate not only modelling but also the ability to communicate results and limitations to a non-technical audience.

10. Limitations, Risks, and Responsible Use

The following limitations materially affect how the results should be interpreted:

- **Small dataset:** approximately three weeks of hourly prices is insufficient to represent many market regimes.
- **Uncertain generalization:** results may change substantially with a different time window, label horizon, or data source.
- **Potential leakage risk:** every feature, label, scaler, and selection step must be checked for future information entering training.
- **Model-selection bias:** repeated experiments on the same short dataset can make the best observed result look stronger than it is.
- **Class imbalance:** the LSTM's strong tendency to predict up illustrates why class distribution and balanced metrics matter.
- **Costs and execution:** simplified costs may not capture spread changes, slippage, latency, rejected orders, and financing charges.
- **Financial risk:** leverage can magnify losses, and historical performance does not guarantee future outcomes.
- **Explainability limits:** feature importance and SHAP describe fitted-model behaviour; they do not prove causality or stable market relationships.
- **Reproducibility:** package versions, seeds, data provenance, model parameters, and full run configurations should be documented.
- **Operational scope:** this is a research/reporting prototype, not a live trading execution platform.

Responsible-use statement

This report is educational and informational. It does not constitute investment advice, a recommendation to buy or sell a currency, or a promise of returns. The model outputs should not be used as the sole basis for financial decisions. Any future live deployment would require independent validation, compliance review where applicable, robust risk limits, monitoring, and explicit human oversight.

11. Recommended Next Steps

Priority	Action	Reason / expected benefit
1 — Critical	Extend EUR/USD history across multiple months and market regimes	Reduce reliance on a very small sample
2 — Critical	Audit label construction, timestamp alignment, and every feature for leakage	Protect the validity of evaluation
3 — Critical	Specify exact entry/exit rules and realistic spread, commission, slippage, and financing assumptions	Make backtest outcomes more economically meaningful
4 — High	Keep a final untouched chronological test set	Reduce model-selection bias
5 — High	Report confusion matrix, precision, recall, F1, balanced accuracy, and class distribution	Explain strengths and failure modes beyond accuracy
6 — High	Add drawdown, volatility, exposure, turnover, expectancy, and risk-adjusted metrics	Evaluate economic risk and return more fully
7 — High	Run paper trading only after robust validation, with risk limits and monitoring	Test operational behaviour without risking capital
8 — Medium	Record Python/package versions, random seeds, configuration, and run timestamps	Improve reproducibility
9 — Medium	Document exact Trading_Signal meanings and dashboard data refresh steps	Make the project easier to review and maintain
10 — Medium	Keep the README, report, workbook, dashboard screenshot, and presentation synchronized	Present consistent project evidence

Suggested acceptance criteria before considering real-world use

- A substantially larger and documented dataset is available.
- No future information leaks into features or preprocessing.
- A final untouched test period shows repeatable results relative to a defensible baseline.
- Backtest results include realistic costs, risk measures, and sensitivity analysis.
- Paper-trading results are monitored over a meaningful period before any deployment decision.
- The system has explicit risk limits, failure handling, logging, and human oversight.

Appendix A. Project File Inventory

The following names are the project files recorded during development. Exact contents and paths should be verified in the local repository before packaging or release.

Folder	Recorded files / purpose
Data	forex_price_data.csv; mc_scenarios.csv; trade_log.csv
Python	data_check.py; forex_analysis.py; forex_analysis_backup.py; random_forest_model.py; random_forest_reduced_features.py; random_forest_fair_comparison.py; xgboost_model.py; xgboost_shap_explainability.py; xgboost_tuning_experiment.py; xgboost_walk_forward.py; xgboost_walk_forward_summary.py; lstm_model.py; model_comparison_report.py; create_excel_workbook.py
Reports	Technical indicators, trading signals, backtest outputs, saved model artifacts, model comparison, tuning and walk-forward reports, feature-importance and SHAP outputs
Excel	Forex_Trading_Signal_AI_Report.xlsx — 12-sheet analytical workbook
PowerBI	Forex_Trading_Signal_AI_Dashboard_Backup.pbix; forex_dashboard.png; Data subfolder with 7 supporting CSVs
Repository	README.md; Git history; public GitHub repository
Presentation	Dedicated folder created for the PowerPoint and PDF presentation exports

Appendix B. Key Terms

Term	Plain-language meaning
Feature	A model input derived from available historical data
Target / label	The outcome the model is trained to predict
Chronological split	A train/test division that respects time order
Holdout set	Data kept aside for evaluation after model fitting
Walk-forward validation	Repeated evaluation on later time windows
Baseline	A simple reference prediction used to judge whether a model adds value
Accuracy	Share of predictions that are correct
Balanced accuracy	Average recall across classes, useful when class counts differ
Feature importance	A measure of how much a model relies on input variables under a chosen method
SHAP	A method for attributing a model prediction to input features
Look-ahead bias	Using information in a simulation that would not have been available at the decision time
Slippage	Difference between expected and actual execution price
Drawdown	A decline from a prior portfolio or strategy peak
Percentage point (pp)	The arithmetic difference between two percentages; 55.56% – 50.56% = 5.00 percentage points

Appendix C. Final Conclusion

The Forex Trading Signal AI Agent project demonstrates a broad analytical workflow—from historical price data and technical features through model comparison, validation, backtesting, explainability, and dashboard reporting. The recorded experiments favour XGBoost among the models tested, but the performance advantage is modest, the sample is small, and the recorded net backtest return is negative. The strongest next step is not to deploy the model, but to improve the data foundation, validation design, economic realism, and reproducibility of the experiment.

Final assessment: useful as a data analytics and machine-learning portfolio project; not yet validated as a profitable or production-ready trading system.

Repository: https://github.com/ram-analyst/Forex_Trading_Signal_AI_Agent